

CyberDefense Pro 2.0

2.5.1 Software Patching and Host Protections

This lesson covers the following topics:

- Operating system, application, and firmware patching
- Patch management
- Patch management considerations
- Patch testing

Operating System, Application, and Firmware Patching

Operating system, application, and firmware patching is updating existing software on a computer or device to fix bugs, address security vulnerabilities, and improve overall performance. The patches may include updates to existing features or introducing entirely new features. The patches are developed by the software developer and released periodically.

The volume of software vulnerabilities and related patches and updates continues to expand ever-increasingly. Managing and applying patches is time-consuming and necessitates a centralized patch management system. Patches are inevitable and apply to various operating systems, application software, cloud instances, and device firmware.

Patch Management

Patch management can be a manual process, an automated process, or a combination of both. Patch automation is essential but often requires occasional manual work to address problems related to deployment or installation errors. For example, administrators might develop custom scripts to help patch management systems more accurately identify missing patches, search for specific hardware information, or gracefully stop and start system services before patching.

An effective patch management strategy requires patch management software to be configured based on the risks associated with each system and its applications. Mission-critical systems must be treated differently than less critical ones to support availability requirements. Desktops are often patched as quickly as possible after a brief testing phase.

Testing, implementation, rollback, and validation are all crucial parts of the patching and configuration management process:

1. Testing consists of verifying the functionality and compatibility of a patch before deployment, as well as any potential risks it poses to the system.
2. After the tests have been conducted, the patch can be implemented on the computer or device.
3. In the event of a problem, the patch can be rolled back to avoid further disruption to the system.
4. Finally, the patch must be validated to ensure it is effective and secure before deployment.

Patch Management Considerations

Important patch management considerations include the following:

- An individual or task-specific team responsible for reviewing vendor-supplied newsletters and security patch bulletins.
- Mechanisms to patch operating systems and all applications running on them, regardless of application vendor.
- Patch management principles that incorporate cloud resources.
- Assigning updates into urgent, important, and non-critical categories.
- A patch test environment where urgent and important patches can be installed, tested, and analyzed before deployment into production.
- Detailed logging designed to support monitoring and troubleshooting of patch deployment activities.
- A method to evaluate firmware updates before deployment.
- Immediate push delivery of critical security patches.
- A routine schedule for the rollout of non-critical patches.

Patch Testing

Patch testing verifies a patch's functionality and compatibility before deploying it on a computer or device. It is an essential part of the patching process, ensuring that the patch does not cause any negative impacts on the system and its components. Patch testing typically consists of simulating the environment that the patch will be deployed to so that the patch can be tested in a secure environment. The results of patch testing provide insight into the stability and security of the patch, allowing the software developer to make necessary adjustments before releasing the patch.

Patch testing aims to determine whether a software patch creates problems with the organization's unique mix of hardware, software, and configuration settings. Patch testing should primarily involve

testing a patch on a single isolated system to determine whether a patch causes problems, such as software crashes or system instability. Additionally, testing should validate those issues addressed by the software patchwork as expected. For example, a patch successfully removes a vulnerability. A common way to test a patch is by setting up a non-production environment hosting like-for-like mission-critical applications, including enterprise applications and networking systems (where available). Doing this allows patches to be deployed by infrastructure teams, validated by software support staff, and assessed by security teams before deployment into the production environment. Additionally, vulnerability scans should verify that patches only resolve vulnerabilities and do not introduce any new ones!

Copyright © CompTIA, Inc. All rights reserved.